

Webapp de Registro Horario con IA

Guia deCodigo

Seminario Azure AI Foundry

Descripcion del sistema

Aplicacion web Python/Flask con autenticacion Microsoft Entra ID y agente conversacional impulsado por el modelo grok-4.3 de Azure AI Foundry. Permite a los empleados fichar, corregir registros y consultar horas trabajadas mediante lenguaje natural.

Flask 3.1 Servidor web	MSAL + Entra ID Autenticacion OAuth2	grok-4.3 / Azure AI Agente conversacional
----------------------------------	--	---

Contenido

1. Vision General de la Aplicacion.....	3
Arquitectura en tres capas.....	3
Flujo completo de una interaccion	3
Tecnologias utilizadas	3
Variables de entorno	3
2. api_client.py — Cliente HTTP de la API REST.....	5
Proposito y justificacion	5
Constante principal.....	5
Patron comun de todas las funciones.....	5
Las 5 funciones disponibles.....	5
Por que se pasa el token a cada llamada	5
3. agent_runner.py — Motor de conversacion con IA.....	6
Por que Chat Completions y no Agents API	6
Constantes de conexion	6
SYSTEM_PROMPT	6
TOOLS — definiciones de herramientas	6
_threads — memoria de conversaciones.....	6
Funciones auxiliares de formateo.....	7
_execute_tool() — ejecucion de herramientas.....	7
send_message() — el bucle ReAct	7
4. app.py — Servidor Flask y autenticacion	8
Configuracion de Entra ID	8
Nota sobre el formato del token.....	8
Flujo OAuth2 Authorization Code	8
Sesion Flask.....	8
Rutas de la aplicacion	8
Decorador login_required	9
_build_msal_app().....	9
5. templates/chat.html — Interfaz de chat	10
Estructura visual.....	10
CSS — puntos clave	10
Comportamiento JavaScript	10
Comunicacion con el servidor.....	10
Integracion con Flask / Jinja2	11
6. Despliegue en Azure App Service.....	12
Variables de entorno en produccion	12
requirements.txt.....	12
URL de produccion.....	12

1. Vision General de la Aplicacion

La webapp es una aplicacion web Python/Flask que permite a los empleados gestionar su registro horario mediante una interfaz de chat conversacional impulsada por IA. El usuario escribe mensajes en lenguaje natural ("ficha mi entrada", "consulta mis horas de esta semana") y el agente IA interpreta la solicitud, llama a la API REST correspondiente y responde en texto.

Arquitectura en tres capas

Capa	Archivo	Responsabilidad
Presentacion	chat.html	Interfaz de chat HTML/CSS/JS. Sin framework.
Aplicacion	app.py	Servidor Flask. Autenticacion Entra ID via MSAL. Gestion de sesion y rutas.
IA / Logica	agent_runner.py	Motor conversacional. Bucle ReAct con grok-4.3. Ejecuta las herramientas de la API.
Acceso a datos	api_client.py	Cliente HTTP. Todas las llamadas a la API FastAPI externa con el token del usuario.

Flujo completo de una interaccion

1. El usuario escribe un mensaje en el chat (chat.html)
2. El frontend lo envia via AJAX a POST /api/message (app.py)
3. app.py recupera el token JWT del usuario de la sesion y llama a agent_runner.send_message()
4. agent_runner envia el mensaje al modelo grok-4.3 junto con las definiciones de 5 herramientas
5. Si el modelo decide usar una herramienta (fichar, consultar...), agent_runner llama a api_client con el token del usuario
6. api_client hace la llamada HTTP a la API FastAPI en Azure con el token del usuario en la cabecera Authorization
7. El resultado vuelve al modelo, que genera la respuesta final en texto
8. La respuesta llega al chat y se muestra al usuario

Tecnologias utilizadas

- Python 3.11 / Flask 3.1 — servidor web
- MSAL (Microsoft Authentication Library) — flujo OAuth2 Authorization Code
- OpenAI SDK 2.41 para Azure OpenAI — acceso al modelo grok-4.3
- Microsoft Entra ID (Azure AD) — autenticacion de usuarios corporativos
- Azure App Service — despliegue en la nube con Oryx build

Variables de entorno

SECRET_KEY	Clave para firmar las cookies de sesion Flask. Valor aleatorio largo en produccion.
CLIENT_SECRET	Secreto del App Registration de la webapp en Entra ID. Solo conocido por el servidor.

REDIRECT_URI	URI de callback OAuth2. http://localhost:5000/callback (local) o https://ws-dignitae-webapp.azurewebsites.net/callback (produccion).
FOUNDRY_API_KEY	API key del recurso Azure AI Foundry para autenticar las llamadas al modelo grok-4.3.

2. api_client.py — Cliente HTTP de la API REST

Primer archivo a estudiar por ser el mas simple. Es la unica pieza que habla directamente con la API FastAPI externa. Todos los demas modulos usan este como intermediario.

Proposito y justificacion

- Encapsula TODAS las llamadas HTTP en un unico lugar
- Centraliza las cabeceras HTTP (Authorization, Content-Type) en la funcion `_headers()`
- Permite cambiar entre entorno local y produccion cambiando solo la constante `API_BASE`
- Si cambia la API REST, solo hay que modificar este archivo — los demas no saben nada de HTTP

Constante principal

```
API_BASE = "https://ws-dignitae.azurewebsites.net"
```

Para pruebas en local se cambia a `http://localhost:8001`. En produccion apunta al Azure App Service de la API.

Patron comun de todas las funciones

Todas las funciones siguen el mismo patron:

1. Reciben el token JWT del usuario como primer parametro
2. Construyen las cabeceras con `_headers(token)` que incluye `Authorization: Bearer <token>`
3. Hacen la llamada HTTP (`requests.get/post`) a la URL correspondiente
4. Devuelven `{"status": codigo_HTTP, "data": cuerpo_JSON}`

El formato de respuesta uniforme permite que `agent_runner.py` interprete el resultado de forma consistente: 200/201 = exito, 401 = token expirado, 409 = conflicto, etc.

Las 5 funciones disponibles

fichar_entrada(token)	POST /fichar-entrada. Registra la entrada ahora mismo. Devuelve 201 OK o 409 si ya hay entrada pendiente.
fichar_salida(token)	POST /fichar-salida. Registra la salida ahora mismo. Devuelve 200 OK, 404 si no hay pendiente, 409 si solapamiento.
corregir_entrada(token, momento, comentario)	POST /corregir-entrada. Registra entrada en hora pasada especifica. El comentario es obligatorio. Devuelve 201 OK.
corregir_salida(token, momento, comentario)	POST /corregir-salida. Busca el registro pendiente mas reciente anterior al momento indicado y lo cierra. Devuelve 200 OK.
consultar_horas(token, fecha_desde, fecha_hasta)	GET /consultar-horas. Devuelve lista de registros y total de horas. Sin fechas = hoy. Devuelve 200 siempre (lista puede ser vacia).

Por que se pasa el token a cada llamada

El token JWT identifica al empleado ante la API FastAPI. La API lo valida en cada peticion y solo devuelve los datos del empleado que firma el token. Esto garantiza que cada usuario solo ve sus propios registros horarios, aunque compartan el mismo servidor webapp.

3. agent_runner.py — Motor de conversacion con IA

El corazon tecnico de la aplicacion. Implementa el agente IA que interpreta el lenguaje natural del usuario y decide que acciones ejecutar en la API.

Por que Chat Completions y no Agents API

Azure AI Foundry ofrece una Agents API de alto nivel que gestiona automaticamente el bucle de herramientas. Sin embargo:

- La Agents API solo es compatible con modelos GPT-* de OpenAI
- El modelo disponible en esta suscripcion MCT es grok-4.3 (xAI / xAI Grok)
- grok-4.3 no es compatible con la Agents API (error: model_not_found)

Solucion: usar la API de Chat Completions con tool_choice="auto", que si es compatible con grok-4.3, e implementar el bucle de herramientas manualmente. El resultado es funcionalmente identico a la Agents API.

Constantes de conexion

```
ENDPOINT = "https://ws-reg-horas-resource.openai.azure.com"
DEPLOYMENT = "grok-4.3"
```

El cliente AzureOpenAI se crea con la API key de FOUNDRY_API_KEY (variable de entorno) y api_version="2025-01-01-preview".

SYSTEM_PROMPT

Instrucciones permanentes enviadas al modelo en cada conversacion. Definen:

- Como comportarse (asistente de registro horario, respuestas concisas en espanol)
- Que herramienta usar en cada situacion (fichar_entrada para "registra mi entrada", etc.)
- Como formatear los resultados de consultar_horas (agrupar por fecha, mostrar totales)

TOOLS — definiciones de herramientas

Lista de 5 objetos JSON Schema que describen cada funcion al modelo. El modelo las recibe en cada llamada y decide autonomamente cual invocar. Cada definicion incluye:

- name: nombre exacto de la funcion en Python
- description: cuando debe usarla el modelo (en lenguaje natural)
- parameters: schema JSON de los argumentos necesarios

Las 5 herramientas corresponden exactamente a las 5 funciones de api_client.py: fichar_entrada, fichar_salida, corregir_entrada, corregir_salida, consultar_horas.

_threads — memoria de conversaciones

Diccionario en memoria que almacena el historial de mensajes de cada conversacion:

```
_threads: dict = {} # { thread_id (UUID) : [mensajes] }
```

Cada lista de mensajes comienza con el SYSTEM_PROMPT y va acumulando los mensajes del usuario, las respuestas del modelo, las llamadas a herramientas y sus resultados. Este historial completo se envia al modelo en cada turno para que tenga contexto de la conversacion.

Limitacion importante: el historial se pierde al reiniciar el servidor. Aceptable para una demo; en produccion se usaria Redis o base de datos.

Funciones auxiliares de formateo

<code>_ts_local(ts_str)</code>	Convierte timestamps UTC de PostgreSQL a hora local +02:00 en formato dd/mm/aaaa HH:MM:SS. Necesario porque la BD almacena en UTC pero los usuarios quieren ver hora local.
<code>_segundos_a_hhmm(seg)</code>	Convierte entero de segundos a formato HH:MM legible (ej: 28800 -> "08:00").
<code>_formatear_consulta(data)</code>	Agrupar la lista plana de registros por fecha local y convierte todos los timestamps. Devuelve {dias: [{fecha, registros}], total_horas_trabajadas}.

`_execute_tool()` — ejecucion de herramientas

Recibe el nombre de la funcion y sus argumentos del modelo, los despacha a `api_client`, e interpreta el resultado HTTP:

- 200/201: exito -> {"success": true, "data": ...}
- 401: token expirado -> {"success": false, "error": "Token expirado..."}
- 409: conflicto -> {"success": false, "error": mensaje_del_API}
- Otros: -> {"success": false, "error": "Error HTTP XXX"}

El wrapper explicito `success:true/false` es clave: sin el, el modelo podria interpretar un 404 (no hay registros) como error en lugar de respuesta valida.

`send_message()` — el bucle ReAct

Funcion principal. Implementa el patron Reason + Act:

1. Crea o recupera el hilo de conversacion del `thread_id`
2. Anade el mensaje del usuario al historial
3. Llama al modelo `grok-4.3` con el historial completo y las definiciones de herramientas
4. Si el modelo responde con `tool_calls` -> ejecuta cada herramienta con `_execute_tool()` y anade el resultado al historial -> vuelve al paso 3
5. Si el modelo responde con texto -> devuelve ese texto como respuesta final al usuario

El bucle puede ejecutar multiples herramientas en secuencia si la solicitud del usuario lo requiere. El modelo decide cuando ha terminado y devuelve su respuesta final.

4. app.py — Servidor Flask y autenticacion

Punto de entrada de la webapp. Orquesta la autenticacion de usuarios y conecta el frontend con el motor de IA.

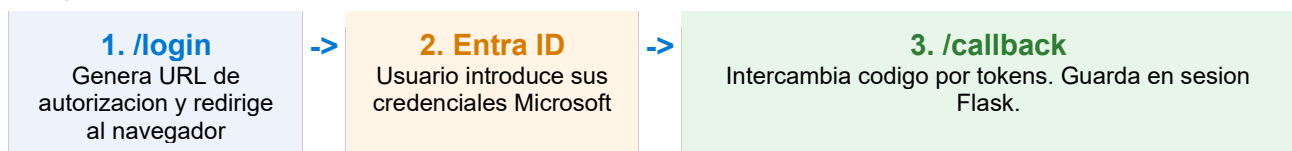
Configuracion de Entra ID

TENANT_ID	Directorio Microsoft Entra ID donde estan registradas ambas apps. Identifica la organizacion.
CLIENT_ID	App Registration de ESTA webapp (cliente-registro-horario). c7f7e1c3-de63-...
API_CLIENT_ID	App Registration de la API FastAPI. 4dd73579-0a62-... Se usa solo para construir el SCOPE.
SCOPES	["api://4dd73579-.../access_as_user"] — permiso delegado que produce el token JWT para la API.
AUTHORITY	https://login.microsoftonline.com/{TENANT_ID} — endpoint de autorizacion de Entra ID.
REDIRECT_URI	URL a la que Entra ID redirige tras el login con el codigo de autorizacion.

Nota sobre el formato del token

El token obtenido via MSAL es de formato v1 (issuer: sts.windows.net) con audience api://4dd73579-... Esto ocurre cuando accessTokenAcceptedVersion en el manifiesto del App Registration de la API no esta fijado a 2. La API FastAPI en auth.py esta configurada para aceptar exactamente este formato.

Flujo OAuth2 Authorization Code



Sesion Flask

Cookie firmada con SECRET_KEY. Contiene tres valores clave:

session["user"]	Claims del id_token: nombre del empleado (campo name) para mostrar en la UI.
session["access_token"]	Token JWT para incluir en las llamadas a la API FastAPI. Identifica al empleado ante la API.
session["thread_id"]	UUID del hilo de conversacion activo en agent_runner._threads. None al inicio y tras reset.

Rutas de la aplicacion

GET /	Ruta raiz. Redirige a /chat si hay sesion activa, o a /login si no.
--------------	---

GET /login	Inicia el flujo OAuth2. Genera URL de autorizacion con MSAL y redirige.
GET /callback	Recibe el codigo de Entra ID. MSAL lo intercambia por tokens. Guarda en sesion.
GET /logout	Limpia sesion Flask y redirige al endpoint de logout de Entra ID (invalida tambien el SSO).
GET /chat	Renderiza chat.html con el nombre del usuario. Protegida por @login_required.
POST /api/message	Recibe {"message": "texto"} del frontend. Llama a agent_runner. Devuelve {"response": "texto"}.
POST /api/reset	Pone thread_id a None en la sesion. La proxima llamada crea un nuevo hilo.

Decorador login_required

Funcion decoradora que protege las rutas privadas. Si no hay clave user en la sesion Flask, redirige automaticamente a /login. Se aplica a /chat, /api/message y /api/reset.

_build_msal_app()

Crea una instancia de ConfidentialClientApplication de MSAL. Se usa ConfidentialClient (y no PublicClient) porque la webapp corre en servidor y puede guardar el CLIENT_SECRET de forma segura. Un cliente publico (ej: app movil) no puede guardar secretos.

5. templates/chat.html — Interfaz de chat

Plantilla Jinja2 que genera una SPA (Single Page Application) de chat. No utiliza ningun framework JavaScript externo — solo HTML, CSS y la API Fetch nativa del navegador.

Estructura visual

Cabecera (header)	Barra azul (#0078D4) con titulo '🕒 Registro Horario', nombre del usuario inyectado por Jinja2 (<code>{{ user_name }}</code>) y enlace 'Cerrar sesion'.
Area de mensajes (#chat-container)	Zona de scroll con burbujas. Mensajes del usuario: azules, alineados a la derecha. Mensajes del asistente: blancos con sombra, alineados a la izquierda.
Barra inferior (#input-area)	Boton '🗨 Nueva conversacion', textarea para escribir y boton 'Enviar'. Fondo blanco con borde superior.

CSS — puntos clave

- Diseno flexbox vertical: body ocupa 100vh, chat-container crece con flex:1
- Burbuja usuario: background #0078D4, color blanco, border-bottom-right-radius:4px (cola derecha)
- Burbuja asistente: background white, box-shadow sutil, border-bottom-left-radius:4px (cola izquierda)
- Burbuja typing: misma forma que asistente pero italic y color #888 para 'Trabajando...'
- white-space:pre-wrap en .message para respetar saltos de linea del modelo

Comportamiento JavaScript

Cuatro funciones principales:

addMessage(text, role)	Crea un div con la clase apropiada (user / assistant / typing), lo anade al chat-container y hace scroll al fondo automaticamente.
sendMessage()	Funcion principal. Lee el textarea, llama a addMessage() para mostrar el mensaje del usuario, muestra 'Trabajando...', hace fetch POST a /api/message y reemplaza el typing con la respuesta.
Enter / Shift+Enter	keydown listener: Enter sin Shift llama sendMessage(). Shift+Enter inserta salto de linea normal en el textarea.
resetBtn click	Llama a POST /api/reset y limpia el chat-container mostrando el mensaje de bienvenida.

Comunicacion con el servidor

Todo el intercambio tras la carga inicial usa AJAX con la API Fetch:

```
fetch("/api/message", { method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message: text }) })
```

El servidor devuelve { "response": "texto" } (o { "error": "msg" } si falla). La pagina nunca se recarga.

Integracion con Flask / Jinja2

El unico punto donde Flask inyecta datos en el HTML es el nombre del usuario:

```
<!-- En Python: render_template('chat.html', user_name=nombre) -->  
<!-- En HTML: {{ user_name }} se reemplaza por el valor -->
```

El resto de la interaccion (mensajes del chat) se gestiona completamente via AJAX, sin volver a renderizar la plantilla.

6. Despliegue en Azure App Service

Variables de entorno en produccion

En local se usan ficheros .env (no subidos al repositorio). En Azure App Service se configuran como Application Settings equivalentes:

SECRET_KEY	Valor aleatorio largo. Nunca el mismo que en local.
CLIENT_SECRET	Secreto del App Registration de Entra ID.
REDIRECT_URI	https://ws-dignitae-webapp.azurewebsites.net/callback
FOUNDRY_API_KEY	API key del recurso Azure AI Foundry.
SCM_DO_BUILD_DURING_DEPLOYMENT	true — activa Oryx build (instala dependencias de requirements.txt automaticamente).

requirements.txt

Dependencias Python necesarias para el despliegue:

```
flask==3.1.0
msal==1.31.0
openai==2.41.0
azure-identity==1.19.0
requests==2.32.3
python-dotenv==1.0.1
gunicorn==23.0.0
```

Gunicorn es el servidor WSGI de produccion. Azure App Service lo usa automaticamente para servir la aplicacion Flask.

URL de produccion

- Webapp: <https://ws-dignitae-webapp.azurewebsites.net>
- API REST: <https://ws-dignitae.azurewebsites.net>
- Documentacion API: <https://ws-dignitae.azurewebsites.net/docs> (Swagger UI)